in a query, with their values readily being available). The typical way to interact with the TSDB is to ask, "In the time frame from *x* to *y*, what are the values or value aggregates matching these conditions?"

The main index used in a TSDB is the time, usually divided into equal buckets. To keep things simple, figure 5.2 uses minute intervals, but usually, the intervals can be as small as seconds or milliseconds in granularity.

Each time series has a unique name (e.g., `mysvc_http_request_total{status_code="400"}`), capturing several dimensions of interest. The dimension of interest here is the HTTP status code returned (`status_code`), and many more, like the HTTP URL path, HTTP method, and so on might also be something you want to track. The metric this time series belongs to is of the type `counter`. That means its values increase strictly monotonically, and it captures the overall number of invocations of the `mysvc` service's HTTP API. In this context, the net change between time periods will give you the number of invocations in a period. A single data point, addressed by time and unique time series, represents the value. In our case, as shown in figure 5.2, it means that at time `2022-08-01T15:04`, the TSDB has seen a total of 31 invocations that resulted in a `400` HTTP status code.

Oftentimes, you want to query a time range, rather than a particular instant in time. You can do this by specifying the start (`2022-08-01T14:58`) and end (`2022-08-01T15:01`). You can, for example, ask the TSDB how many invocations of a certain type occurred; in our case, if you're interested in client errors, you'd ask how many `400` HTTP status codes were witnessed in the time frame, and the answer would be 12 (at the end of range, we have 24, and at the start, we had 12).

Cardinality explosion

You might wonder what happens if you start adding dimensions to a metric you want to track, with a metric being the umbrella term for all related time series. In this example, let's say a dimension you're interested in is the user ID.

In contrast to, say, the HTTP status code, which has a small, bounded number of values (a couple of dozens, from 1xx to 5xx), the user ID could have many values. If your app is successful, potentially serving many hundreds of thousands of users, this means the TSDB has to ingest and store hundreds of thousands of time series (for that single metric only, and you may have thousands of metrics you want to track).

This idea of many different values a dimension can take on being multiplied by the number of metrics is what we call *cardinality explosion* and can result in slow query times, since the TSDB needs to inspect each time series in a given time range, for each dimension. That systems property makes TSDBs for certain types of observability use cases not the best fit: at the very least, you should be aware of this and avoid adding dimensions that you know or suspect of "exploding," in terms of their cardinality.

If you would like to dive deeper into the topic of TSDBs and their use cases in general, consider reading the article "What Is Time-Series Data? Definitions & Examples" by Ajay